

7. Statsmodels及其基本使用

前面的pandas和scipy已经为我们提供了很好的统计分析框架了。这里我们介绍一个进阶的统计分析工具也就是statsmodels。这个工具主要是进行回归及其扩展，还可以进行时间序列的预测，用法相对而言并不是那么多。本章主要包括：

- [7.1 方差分析与回归分析](#)
- [7.2 时间序列预测](#)

7.1 方差分析与回归分析

7.1.1 方差分析

在统计学中，方差分析（ANOVA, Analysis of Variance）是一种用于比较两个或多个样本均值的统计方法，通过检验样本间是否存在显著性差异来判断这些样本是否来自同一总体。在Python中，statsmodels库提供了进行方差分析的功能，主要使用ols（普通最小二乘法）模型进行线性回归后，再应用方差分析的工具。然而，对于直接的方差分析，statsmodels提供了anova_lm函数，该函数可以直接对线性模型的结果进行方差分析。

以下是一个如何使用statsmodels进行单因素方差分析（One-Way ANOVA）的基本示例：

1. **准备数据**：首先，你需要有一组数据，这些数据可以根据某个分类变量被分为不同的组。
2. **导入必要的库**：

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

3. **构建数据**：这里我们使用pandas的DataFrame来模拟一些数据。

```
# 假设有三组数据，每组数据代表不同的处理方式
data = {'Group': ['A']*10 + ['B']*10 + ['C']*10,
        'Value': np.random.normal(0, 1, 10) + 1 + np.random.normal(0, 1, 10) + 2 + np.random.no
df = pd.DataFrame(data)
```

注意：这里为了演示效果，我们人为地给每组数据加上了不同的均值（1, 2, 3），并添加了随机噪声。

4. 使用 `ols` 模型并进行方差分析：

虽然 `ols` 本身用于线性回归，但我们可以利用它来拟合数据，并通过 `anova_lm` 进行方差分析。

```
# 使用公式API定义模型，其中C(Group)表示将Group作为分类变量
model = ols('Value ~ C(Group)', data=df).fit()

# 使用anova_lm进行方差分析
anova_results = sm.stats.anova_lm(model, typ=1) # typ=1表示I型平方和

print(anova_results)
```

这里的 `C(Group)` 告诉 `ols` 将 `Group` 视为分类变量。 `anova_lm` 函数计算了ANOVA表，其中 `typ` 参数指定了平方和的类型（I型、II型或III型）。

5. 解读结果：

`anova_results` 将包含每个分类变量的F统计量、P值等信息。你可以根据P值来判断不同组之间是否存在显著性差异。如果P值小于显著性水平（通常是0.05），则认为这些组之间存在显著性差异。

请注意，以上示例是基于单因素方差分析（即只有一个分类变量）。如果你需要进行多因素方差分析（ANOVA with multiple factors），你可以在模型中加入更多的分类变量，并使用相同的流程进行分析。不过，对于复杂的实验设计，可能还需要考虑交互作用等因素。

7.1.2 回归分析

在Python中，使用 `statsmodels` 库进行线性回归是一种非常常见且强大的数据分析方法。下面我将给出一个简单的线性回归代码案例，用于展示如何使用 `statsmodels` 来拟合线性模型。

接下来是线性回归的代码案例：

```

import numpy as np
import statsmodels.api as sm
import pandas as pd

# 示例数据
# 假设我们有一组自变量X和因变量Y
X = np.array([1, 2, 3, 4, 5]) # 自变量
Y = np.array([2, 3, 5, 7, 11]) # 因变量, 这里  $Y = 1.4 * X + 0.5 + \text{noise}$ , 其中noise为随机噪声

# 为了使用statsmodels的OLS（普通最小二乘法）模型，我们需要将X转换为二维数组，
# 并添加一个常数项以拟合截距
X = sm.add_constant(X.reshape(-1, 1)) # 添加常数项，reshape(-1, 1)将X转换为二维数组

# 拟合模型
model = sm.OLS(Y, X).fit()

# 打印模型摘要
print(model.summary())

# 如果你想直接获取模型的参数（系数和截距），可以这样做：
print("模型参数: ", model.params)
# 输出中，第一个值是截距，第二个值是X的系数

# 使用模型进行预测
X_new = np.array([6, 7]).reshape(-1, 1) # 新数据点
X_new = sm.add_constant(X_new) # 同样需要添加常数项
predictions = model.predict(X_new)
print("预测值: ", predictions)

```

在这个例子中，我们首先生成了一组简单的自变量 x 和因变量 y 的数据，其中 y 是 x 的线性函数加上一些随机噪声。然后，我们使用 statsmodels 的 OLS 类来拟合这些数据。在拟合之前，我们通过在 x 中添加一个常数项来准备数据，这是为了模型能够拟合出截距项。

`fit()` 方法用于拟合模型，而 `summary()` 方法则提供了一个模型的详细摘要，包括模型的R-squared值、每个系数的统计量和p值等重要信息。

最后，我们使用拟合好的模型对新的数据点 `X_new` 进行预测，并打印出预测结果。

请注意，为了使用 `predict()` 方法进行预测，新数据点 `X_new` 也需要被转换为二维数组，并且同样需要添加常数项。

在 statsmodels 中，广义线性模型（Generalized Linear Models, GLM）是一种灵活的统计模型，用于估计因变量的条件均值，该因变量与自变量之间的关系通过链接函数（link function）和指数族分布（exponential family distributions）来指定。GLM 扩展了普通线性模型（OLS），允许因变量遵循非正态分布（如二项分布、泊松分布等），并且因变量的期望值可以通过非线性链接函数与自变量线性相关。

以下是如何使用 statsmodels 进行广义线性模型（GLM）的一个基本示例：

首先，确保你已经安装了 statsmodels 库。如果没有安装，可以通过pip安装：

```
pip install statsmodels
```

然后，你可以使用以下代码来拟合一个GLM：

```

import numpy as np
import statsmodels.api as sm

# 示例数据
# 自变量X和因变量Y（这里Y是二项分布的数据，例如成功/失败的次数）
X = np.random.rand(100, 2) # 生成100个样本，每个样本2个特征
# 假设真实的模型是  $\text{logit}(p) = 0.5 + 1.5 \cdot X[:,0] - 0.8 \cdot X[:,1]$ ，其中p是成功的概率
p = 1 / (1 + np.exp(-(0.5 + 1.5*X[:,0] - 0.8*X[:,1])))
Y = np.random.binomial(n=1, p=p) # 根据p生成二项分布的因变量

# 为了使用GLM，我们需要将Y转换为“成功”的次数（在这里，成功就是Y=1的情况）
# 对于二项分布，我们还需要知道每次试验的次数（这里是1），但GLM在statsmodels中通常只接受成功次数
# 注意：对于二项GLM，我们实际上只需要Y（成功次数）和n（试验次数）的数组，但statsmodels的GLM接口只接受
# 如果n不是固定的，则需要以不同的方式处理（例如，使用权重或转换为其他形式的数据）

# 添加常数项以拟合截距
X = sm.add_constant(X)

# 拟合GLM模型，这里使用logit链接函数和二项分布
model = sm.GLM(Y, X, family=sm.families.Binomial())
result = model.fit()

# 打印模型摘要
print(result.summary())

# 预测新数据点的概率
X_new = np.array([[0.2, 0.3], [0.8, 0.1]])
X_new = sm.add_constant(X_new)
predicted_probs = result.predict(X_new, linear=False) # linear=False给出预测的概率，而非线性预测值
print("预测的成功概率: ", predicted_probs)

```

在这个例子中，我们生成了一组模拟数据，其中因变量 Y 是二项分布的，表示成功或失败的次数（在这个案例中，试验次数 n 固定为1）。然后，我们使用 `statsmodels` 的 `GLM` 类来拟合一个GLM模型，其中指定了 `Binomial` 分布和 `logit` 链接函数。拟合完成后，我们打印了模型的摘要，并使用拟合好的模型来预测新数据点的成功概率。

注意，在调用 `predict` 方法时，`linear=False` 参数表示我们想要得到的是通过链接函数反变换后的预测概率，而不是线性预测值。如果 `linear=True`（默认值），则 `predict` 方法将返回线性预测值，这些值需要通过链接函数的反函数来转换为概率或其他适当的度量。

7.1.3 方差分析与回归分析是统一的框架

线性回归和方差分析 (Analysis of Variance, ANOVA) 虽然在统计方法和应用场景上有所不同, 但它们在某些方面共享着相似的逻辑和框架, 尤其是在处理变量之间的关系和差异时。它们各自有独特的目标、假设和适用场景。不过, 可以从以下几个方面来理解它们之间的联系:

1. 模型基础

- i. **线性关系**: 线性回归是基于自变量 (解释变量) 和因变量之间线性关系的假设, 通过最小二乘法等方法来拟合这种关系。方差分析虽然不直接建立自变量和因变量之间的线性模型, 但它也隐含了变量之间可能存在的某种关系或差异, 这种关系或差异可以通过比较不同组之间的均值来揭示。
- ii. **统计推断**: 两者都涉及统计推断, 即利用样本数据来推断总体特征。线性回归通过估计回归系数和进行假设检验来推断自变量对因变量的影响; 方差分析则通过比较不同组之间的均值差异来推断某个因素 (自变量) 对观测值 (因变量) 的影响是否显著。

2. 假设条件

- i. **独立性**: 两者都要求观测值之间相互独立, 即一个观测值不影响其他观测值。
- ii. **正态性**: 虽然线性回归对误差项的正态性有明确要求 (误差项服从均值为0的正态分布), 但方差分析在某些情况下 (如单因素方差分析) 对原始数据的正态性要求不是非常严格, 更关注于组间差异的显著性。然而, 在更复杂的方差分析 (如多因素方差分析) 中, 也会考虑正态性和同方差性等假设条件。

3. 应用场景

- i. **线性回归**: 主要用于预测和解释, 通过建立自变量和因变量之间的线性关系模型, 可以预测因变量的值或解释自变量对因变量的影响程度。
- ii. **方差分析**: 主要用于比较和推断, 通过比较不同组之间的均值差异来推断某个因素 (自变量) 对观测值 (因变量) 的影响是否显著。它常用于实验设计、社会科学研究等领域。

4. 结合使用

在某些情况下, 线性回归和方差分析可以结合使用以提供更全面的数据分析结果。例如, 在方差分析得到群体均值差异显著的情况下, 可以进一步使用线性回归来确定差异的原因和自变量与因变量之间的具体关系。

7.2 时间序列预测

7.2.1 时间序列分解

`seasonal_decompose` 是 `Statsmodels` 库中用于时间序列分解的一个非常有用的函数, 它可以帮助我们将时间序列数据分解为趋势 (Trend)、季节性 (Seasonality) 和残差 (Residual) 三个部分。下面将详

详细介绍 `seasonal_decompose` 的用法，并给出几个案例。

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

`seasonal_decompose` 函数的主要参数包括：

- `x`：被分解的时间序列数据，通常是一个 `pandas Series` 或类似数组结构的对象。
- `model`：分解模型，可以是 `'additive'`（加法模型）或 `'multiplicative'`（乘法模型），默认为 `'additive'`。
- `period`：时间序列的周期长度，如果 `x` 是 `pandas` 对象且其索引有频率，则可以省略此参数。
- `filt`：可选，用于滤除季节性成分的滤除系数，默认为 `None`。
- `two_sided`：滤波中使用的移动平均法，如果为 `True`（默认），则使用 `filt` 计算居中的移动平均线。
- `extrapolate_trend`：如果设置为 `> 0`，则考虑到许多 `(+1)` 最接近的点，由卷积产生的趋势将在两端外推线性最小二乘法。如果设置为 `'freq'`，则使用频率最近点。

`seasonal_decompose` 返回一个 `DecomposeResult` 对象，该对象包含分解出的趋势、季节性和残差成分等信息。

案例 1：饮料产品销售数据分析

假设我们有一个关于某饮料产品在上海地区2022年1月至2023年3月期间每日销售量的时间序列数据。

```
# 假设 df 是包含日期 ('date') 和销售量 ('sales') 的 pandas DataFrame
# 首先，我们需要将 'date' 列设置为索引，并确保它是时间序列索引
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)

# 使用 seasonal_decompose 进行分解
result = seasonal_decompose(df['sales'], model='additive', period=7)

# 绘制分解结果
result.plot()
plt.show()
```

在这个案例中，我们将周期设置为7天，因为饮料销售通常受到周末和工作日的影响，具有周季节性。

案例 2：月度销售数据分析

假设我们有一个关于某商品在一年中各个月份销售量的时间序列数据。

```
# 假设 df 是包含月份 ('month') 和销售量 ('sales') 的 pandas DataFrame
# 我们需要先将月份转换为时间序列索引 (这里为了简化, 假设月份是从1到12的连续整数)
# 在实际应用中, 可能需要使用 pd.date_range 或 pd.to_datetime 与 pd.PeriodIndex 来创建时间序列索引

# 这里我们直接跳过创建时间序列索引的步骤, 直接分解
# 注意: 在实际应用中, 你应该先确保 'month' 列是正确的时间序列索引
# result = seasonal_decompose(df['sales'], model='additive', period=12)
# 绘制分解结果
# result.plot()
# plt.show()

# 由于直接操作月份索引可能较为复杂, 这里仅提供代码框架
```

注意: 在这个案例中, 由于缺少具体的数据和完整的索引设置, 我省略了直接分解的步骤。在实际应用中, 你需要先将月份数据转换为时间序列索引, 并设置正确的周期 (在这个例子中是12个月)。

注意:

- 在使用 `seasonal_decompose` 之前, 请确保你的时间序列数据是完整的, 没有缺失值。如果数据中存在缺失值, 你可能需要先进行插值或删除这些缺失值。
- 选择合适的模型 (加法模型或乘法模型) 对于分解结果的质量至关重要。通常, 如果时间序列中的趋势和季节性变化是累加的, 则使用加法模型; 如果它们呈现出指数增长或衰减的趋势, 则使用乘法模型。
- 周期长度的选择也应该基于数据的实际情况。对于日数据, 周期可能是7天 (一周); 对于月数据, 周期可能是12个月; 等等。

7.2.2 ARIMA模型

在Python的 `statsmodels` 库中, ARIMA (自回归积分滑动平均模型) 是一种常用的时间序列预测模型。ARIMA模型结合了自回归 (AR)、差分 (I) 和移动平均 (MA) 三个部分来预测未来值。以下是如何在 `statsmodels` 中使用ARIMA模型进行时间序列预测的详细步骤和代码案例。

首先, 你需要导入 `pandas` 用于数据处理, `matplotlib.pyplot` 用于绘图, 以及 `statsmodels.tsa.arima.model.ARIMA` 用于建立ARIMA模型。

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
import numpy as np
```


加载时间序列数据，并将其转换为适合ARIMA模型的形式。数据需要是平稳序列，所以，在进行时间序列预测之前要进行平稳性检验。这里我们一般使用ADF单位根检验。

```
# 假设你有一个CSV文件，其中包含时间序列数据
df = pd.read_csv('time_series_data.csv', index_col='Date', parse_dates=True)
ts = df['Value'] # 假设时间序列数据在 'Value' 列

# 检查平稳性 (ADF测试)
result = adfuller(ts)
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])

# 如果p-value很大，则时间序列可能不是平稳的，需要进行差分
ts_diff = ts.diff().dropna()

# 再次检查差分后的平稳性 (可选)
result_diff = adfuller(ts_diff)
print('ADF Statistic: %f' % result_diff[0])
print('p-value: %f' % result_diff[1])
```

一旦你的时间序列是平稳的（或你认为它足够接近平稳），你就可以使用ARIMA模型进行拟合了。你需要指定模型的参数(p, d, q)，其中p是自回归项的阶数，d是差分阶数（已经通过步骤2的差分操作隐含指定），q是移动平均项的阶数。

```
# 假设我们已经通过某种方式（如ACF和PACF图）选择了p, d, q
# 注意：这里的d=1是因为我们在步骤2中已经进行了一次差分
p = 1
d = 1
q = 1

# 拟合ARIMA模型
model = ARIMA(ts, order=(p, d, q))
fit_model = model.fit()

# 打印摘要信息
print(fit_model.summary())
```

使用拟合好的模型进行未来值的预测。

```
# 预测未来5个时间点的值
forecast = fit_model.forecast(steps=5)
print(forecast)

# 绘制原始数据和预测数据
plt.figure(figsize=(10, 5))
plt.plot(ts.index, ts, label='Original')
plt.plot(ts.index[-1:] + pd.DateOffset(days=1):ts.index[-1:] + pd.DateOffset(days=6), forecast,
plt.legend()
plt.show()
```

注意：在上面的预测代码中，我使用了 `ts.index[-1:] + pd.DateOffset(days=x)` 来生成未来时间点的索引，其中 `x` 是从1到5的整数。这是为了将预测结果绘制在正确的日期上。但是，这假设了时间序列是日数据，并且每天都有一个观测值。如果你的数据频率不同（如月度、季度等），你需要相应地调整索引的生成方式。

注意：

- 在实际应用中，选择正确的ARIMA模型参数（`p, d, q`）是关键。这通常涉及到查看时间序列的ACF（自相关函数）和PACF（偏自相关函数）图，或者使用网格搜索等优化方法来找到最佳参数。
- 有时，对数据进行季节性差分或转换（如取对数）可能是必要的，以进一步改善模型的性能。
- 始终关注模型的诊断统计量和残差图，以确保模型没有遗漏重要的信息或存在其他问题。